

WDJ UNIT 2 QB ANS (2- marks)

Some questions have asked only for code still I have added answer for understanding

1. Differentiate between GET and POST

1. **GET method** is used to retrieve data from the server. **POST method** is used to send data from the client to the server.
2. In **GET**, data is sent in the URL as parameters. In **POST**, data is sent inside the request body.
3. In **GET**, the data is visible in the browser address bar. In **POST**, the data is not visible in the URL.
4. **GET** has a limited data size because it uses the URL. **POST** allows sending larger amounts of data.

2. What is session tracking?

1. **Session tracking** is a technique used to maintain information about a user during multiple requests. It helps the server remember the user's previous activities.
2. HTTP is a **stateless protocol**, which means it cannot remember earlier requests. Session tracking helps maintain a continuous interaction between client and server.
3. It is commonly used in applications like **shopping carts and login systems**. It allows the server to identify the same user across multiple pages.
4. Session tracking can be implemented using techniques such as **HttpSession and cookies**. These methods help store and retrieve user-related information.

3. What is HttpSession?

1. **HttpSession** is an interface used in servlets to manage user sessions. It allows the server to store information about a user during a session.
2. A session object is created using the **getSession() method** of `HttpServletRequest`. If a session already exists, the same session object is returned.
3. Data can be stored in a session using the **setAttribute() method**. The stored data can be retrieved later using the **getAttribute() method**.
4. HttpSession helps maintain user information across multiple requests. It is commonly used for login sessions and user tracking.

4. What is a cookie?

1. A **cookie** is a small text file stored on the client's computer by a website. It is used to store information about the user.
2. Cookies help maintain state between multiple requests in web applications. They allow the server to recognize returning users.
3. Cookies are created by the server and stored in the browser. The browser sends them back to the server in future requests.
4. Cookies are commonly used for **session tracking and user preferences**. They help personalize the user experience on websites.

5. How is a cookie created in servlet?

1. A cookie is created using the **Cookie class** in the servlet API. The constructor `Cookie(String name, String value)` is used to create a cookie object.
2. After creating the cookie, it is sent to the client using the **`response.addCookie()`** method. This stores the cookie in the user's browser.
3. The browser saves the cookie on the client computer. It automatically sends the cookie back to the server in future requests.
4. The servlet can read stored cookies using the **`request.getCookies()`** method. This helps retrieve information saved in the cookie.

6. List any two differences between `ServletConfig` and `ServletContext`

1. **`ServletConfig`** is used for configuration of a specific servlet. **`ServletContext`** is used for configuration of the entire web application.
2. There is **one `ServletConfig` object per servlet**. There is **one `ServletContext` object per application**.
3. **`ServletConfig` parameters** are available only to that servlet. **`ServletContext` parameters** can be shared by all servlets in the application.
4. `ServletConfig` is mainly used to read **initialization parameters for a servlet**. `ServletContext` is used to **share data and resources across the application**.

7. Define session in web applications

1. A **session** is a continuous interaction between a user and a web application. It starts when a user logs in or begins using the application.
2. A session continues until the user logs out or closes the browser. It allows the server to keep track of the user during multiple requests.
3. Sessions are required because **HTTP is a stateless protocol**. Without sessions, the server cannot remember previous requests from the same user.
4. Sessions are commonly used in applications like **online shopping and login systems**. They help store temporary user information during interaction.

8. Explain the purpose of ServletConfig

1. **ServletConfig** is an interface used to configure a specific servlet. It provides initialization parameters for that servlet.
2. A **ServletConfig object is created by the servlet container** during servlet initialization. It stores configuration data defined in the web.xml file.
3. It allows a servlet to read initialization parameters using the **getInitParameter()** method. These parameters are used to configure servlet behavior.
4. ServletConfig parameters belong only to a **single servlet and cannot be shared** with other servlets. It helps manage servlet-specific configuration.

9. Explain the purpose of ServletContext

1. **ServletContext** is an interface used to share information across the entire web application. It allows multiple servlets to access common data.
2. Only **one ServletContext object is created for each web application**. It is created when the application starts and destroyed when it stops.
3. ServletContext allows storing and retrieving data using **setAttribute() and getAttribute() methods**. This helps share information between different servlets.
4. It can also read **application-wide initialization parameters** from the web.xml file. These parameters are common for the entire application.

10. Write methods used to retrieve request parameters in a servlet

1. The **getParameter(String name)** method is used to retrieve the value of a request parameter. It returns the value as a string.
2. The **getParameterNames()** method is used to get all parameter names from the request. It returns the parameter names as an Enumeration.
3. The **getParameterValues(String name)** method is used when a parameter has multiple values. It returns an array of values for that parameter.
4. These methods are available in the **ServletRequest** or **HttpServletRequest** **interface**. They help the servlet read data sent by the client.

11. Which method is used to create a session in servlet?

1. The **getSession()** **method** is used to create or retrieve a session in a servlet. It is available in the **HttpServletRequest** interface.
2. If a session already exists for the client, the same session object is returned. If not, a new session is created.
3. The method can also be used as **getSession(true)** to create a new session if none exists. It helps maintain user information across requests.
4. The created session object is of type **HttpSession**. It allows storing and retrieving user data during a session.

12. How do you add a cookie to response?

1. A cookie object is created using the **Cookie class constructor**. It requires a name and value for the cookie.
2. After creating the cookie, it is added to the response using **response.addCookie(cookie)**. This sends the cookie to the client browser.
3. The browser stores the cookie on the client computer. It sends the cookie back to the server in future requests.
4. This method is useful for **session tracking and storing user preferences**. It helps maintain state in web applications.

13. Which method is used to handle GET request in HttpServlet?

1. The **doGet() method** is used to handle GET requests in HttpServlet. It is called when a client sends a GET request to the server.
2. The method receives **HttpServletRequest** and **HttpServletResponse** objects as parameters. These objects help process the request and send a response.
3. Developers override the **doGet() method** to define how the servlet should respond to GET requests. It usually retrieves and displays data.
4. GET requests are commonly used for **retrieving web pages or searching data**. The response generated by doGet() is sent back to the browser.

(5 – marks)

1. Explain various HTTP request methods

1. **HTTP request methods** define the type of action the client wants the server to perform. These methods are used for communication between a web browser and a web server.
2. The **GET method** is used to retrieve data from the server. The data is sent in the URL and is visible in the browser address bar.
3. The **POST method** is used to send data to the server for processing. The data is sent inside the request body and is not visible in the URL.
4. The **PUT method** is used to update or replace an existing resource on the server. It sends the complete updated data to the server.
5. The **DELETE method** is used to remove a resource from the server. It is commonly used in web services and REST APIs.
6. Other methods like **HEAD, OPTIONS, PATCH, and TRACE** are also available in HTTP. These methods are used for specific purposes such as retrieving headers or checking supported operations.

2. Write a servlet program to handle GET request and display user input

1. A servlet can handle a GET request using the **doGet() method** of the `HttpServlet` class. This method receives the request from the browser and processes it on the server.
2. The user input is usually collected using an **HTML form with method="get"**. When the form is submitted, the data is sent to the servlet through the URL parameters.
3. The servlet retrieves the user input using the **getParameter() method** from the `HttpServletRequest` object. This method returns the value entered by the user.
4. The servlet sets the response type using **response.setContentType("text/html")**. Then a `PrintWriter` object is used to send output back to the browser.
5. The servlet prints a message or displays the user input on the web page. The response is generated in HTML format so that it can be displayed in the browser.
6. Example code:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class GreetServlet extends HttpServlet {
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String name = request.getParameter("username");
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<html><body>");
        out.println("<h2>Welcome " + name + "</h2>");
        out.println("</body></html>");
    }
}
```

This servlet receives the username from the form and displays a welcome message in the browser.

3. Develop a servlet to demonstrate session tracking using HttpSession

1. **Session tracking** is used to maintain information about a user across multiple requests. This is necessary because HTTP is a stateless protocol and cannot remember previous requests.
2. In servlets, session tracking can be implemented using the **HttpSession interface**. A session object is created or retrieved using the `request.getSession()` method.
3. The servlet can store user information in the session using the **setAttribute() method**. This allows the server to remember user details during the session.
4. When the user sends another request, the servlet retrieves stored data using the **getAttribute() method**. This helps maintain continuity during interaction with the application.
5. Session tracking is commonly used in applications such as **online shopping carts, login systems, and personalized web pages**. It allows the server to identify a specific user during multiple requests.
6. Example code:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class SessionServlet extends HttpServlet {
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        HttpSession session = request.getSession();
        session.setAttribute("username", request.getParameter("name"));

        PrintWriter out = response.getWriter();
        out.println("User Name: " + session.getAttribute("username"));
    }
}
```

4. Write a servlet program to demonstrate cookies usage

1. **Cookies** are small pieces of data stored on the client browser by the server. They are used to store information and maintain user state between requests.
2. In a servlet, a cookie is created using the **Cookie class constructor** with a name and value. The cookie is then sent to the client using the `response.addCookie()` method.
3. The browser stores the cookie and sends it back to the server in future requests. The servlet can read cookies using the `request.getCookies()` method.
4. Cookies help maintain user preferences and session information. They are commonly used for login tracking and personalization.
5. Example code:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class CookieServlet extends HttpServlet {
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        Cookie ck = new Cookie("username","Meet");
        response.addCookie(ck);

        PrintWriter out = response.getWriter();
        out.println("Cookie Created Successfully");
    }
}
```


5. Create a servlet using ServletConfig to read initialization parameters

1. **ServletConfig** is an interface used to provide configuration information to a specific servlet. It allows the servlet to read initialization parameters defined in the deployment descriptor.
2. A ServletConfig object is created by the servlet container during servlet initialization. It stores parameters defined in the **web.xml file** using <init-param>.
3. The servlet can read these parameters using the **getInitParameter() method**. These values are used to configure the behavior of that servlet.
4. ServletConfig parameters are specific to one servlet. They cannot be shared with other servlets in the application.
5. Example code:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class ConfigServlet extends HttpServlet {
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws IOException {

        ServletConfig config = getServletConfig();
        String email = config.getInitParameter("adminEmail");

        PrintWriter out = response.getWriter();
        out.println("Admin Email: " + email);
    }
}
```

This servlet reads an initialization parameter using ServletConfig.

6. Explain session tracking techniques in servlet

1. **Session tracking** is a technique used to maintain user information across multiple requests. It helps the server remember user actions during interaction with a web application.
2. One common technique is **HttpSession**, where a session object is created using `request.getSession()`. It allows storing user data using `setAttribute()` and retrieving it using `getAttribute()`.
3. Another technique is **Cookies**, which store small pieces of data in the user's browser. The browser sends the stored cookies back to the server with each request.
4. **URL rewriting** is another technique where session information is added to the URL. The server reads the session data from the URL parameters.
5. **Hidden form fields** can also be used to store session data inside HTML forms. These fields send session information along with form submissions.
6. These techniques help maintain user state in web applications like **shopping carts, login systems, and personalized services**.

7. Briefly explain the differences between ServletConfig and ServletContext object

1. **ServletConfig** is used to configure a specific servlet in a web application. **ServletContext** is used to provide configuration and information for the entire web application.
2. There is **one ServletConfig object for each servlet** created by the container. There is **only one ServletContext object for the entire application**.
3. **ServletConfig parameters** are defined using `<init-param>` in the web.xml file. **ServletContext parameters** are defined using `<context-param>` in the web.xml file.
4. Data stored in **ServletConfig** can only be accessed by that particular servlet. Data stored in **ServletContext** can be shared by all servlets in the application.
5. ServletConfig is mainly used for **servlet-specific initialization parameters**. ServletContext is used for **application-wide data sharing and resource access**.

8. Explain HttpServletRequest with its methods

1. **HttpServletRequest** is an interface that represents the client request sent to the servlet. It contains information about request parameters, headers, and client data.
2. The **getParameter(String name)** method is used to retrieve the value of a parameter sent by the client. It returns the parameter value as a string.
3. The **getParameterNames()** method returns all parameter names sent in the request. It provides them in the form of an Enumeration.
4. The **getParameterValues(String name)** method returns multiple values of a parameter if it appears more than once. It returns the values as an array of strings.
5. HttpServletRequest also provides other methods to obtain request information such as headers, session data, and cookies. These methods help the servlet process client requests effectively.

9. Explain HttpSession in detail

1. **HttpSession** is an interface used to manage sessions in a web application. It helps maintain information about a user during multiple requests.
2. A session object is created or retrieved using the **request.getSession() method**. If a session already exists, the same session is returned; otherwise a new session is created.
3. Data can be stored in a session using the **setAttribute(String name, Object value)** method. The stored data can later be retrieved using the **getAttribute(String name)** method.
4. HttpSession provides useful methods such as **getId()**, **getCreationTime()**, and **getLastAccessedTime()**. These methods help manage and track session information.
5. HttpSession is commonly used in applications like **login systems and online shopping carts**. It allows the server to maintain user state across multiple web requests.

10. Write a short note on ServletContext interface

1. **ServletContext** is an interface that provides information about the web application environment. It allows servlets to communicate with the servlet container.
2. There is **only one ServletContext object for each web application**. It is created when the application starts and destroyed when the application stops.
3. ServletContext allows sharing data among different servlets using **setAttribute() and getAttribute() methods**. This helps multiple servlets access common information.
4. It can also read **application-wide initialization parameters** defined in the web.xml file using the `getInitParameter()` method. These parameters are available to all servlets.
5. ServletContext also provides methods for accessing resources and forwarding requests to other components. It helps manage application-level configuration and data sharing.

11. What are Cookies? Explain how Servlets store and read data from Cookies

1. **Cookies** are small text files stored on the client computer by a website. They are used to store user information and maintain state between requests.
2. In servlets, cookies are created using the **Cookie class constructor** with a name and value. The cookie is then sent to the browser using the `response.addCookie()` method.
3. The browser stores the cookie and sends it back to the server with future requests. This allows the server to remember information about the user.
4. Servlets can read cookies using the **request.getCookies() method**. This method returns an array of cookies stored in the browser.
5. The servlet can access each cookie using **getName() and getValue() methods**. This helps retrieve stored information such as user preferences or login data.

12. Explain cookies in servlet with advantages and disadvantages

1. **Cookies** are small text files stored on the client computer by a website. They are used to store user information and maintain state between requests in web applications.
2. In servlets, cookies are created using the **Cookie class** with a name and value. The cookie is sent to the browser using the `response.addCookie()` method and later retrieved using `request.getCookies()`.
3. One advantage of cookies is that they help **maintain session information and user preferences**. They also improve user experience by remembering settings and login information.
4. Another advantage is that cookies are **simple to implement and require less server memory**. They store data on the client side instead of the server.
5. One disadvantage is that cookies **have size limitations and security risks**. Users can also disable cookies in the browser, which may affect the application functionality.

13. Explain JDBC connectivity using servlet with example

1. **JDBC (Java Database Connectivity)** is used to connect Java applications with a database. In servlets, JDBC is used to retrieve or store data from a database.
2. The first step is to **load the JDBC driver** using `Class.forName()`. This allows the servlet to communicate with the database system.
3. The second step is to **establish a connection** using `DriverManager.getConnection()` with database URL, username, and password. This creates a connection between the servlet and database.
4. After creating the connection, a **PreparedStatement or Statement** object is used to execute SQL queries. The results are retrieved using a **ResultSet** object.
5. Example code:

```
import java.io.*;
import java.sql.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class LoginServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        PrintWriter out = response.getWriter();
        String user = request.getParameter("username");

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/college","root","password");

            PreparedStatement ps = con.prepareStatement(
                "SELECT * FROM student WHERE name=?");
            ps.setString(1, user);

            ResultSet rs = ps.executeQuery();
            if(rs.next())
                out.println("Login Successful");
            else
                out.println("Invalid User");

            rs.close();
            ps.close();
            con.close();
        } catch (Exception e) {
            out.println(e);
        }
    }
}
```